Who Wrote It?

Identifying LLM Families from Their Responses

Kevin Coulter

Individual Project

Pennsylvania State University

CMPSC 448: Machine Learning

Midterm Project Report

October 11, 2026

1 Problem Definition and Dataset Curation

Large language models may produce recognizable patterns in their wording, formatting, and response structure. This project tests whether neural networks can use those patterns to identify a response's originating model family. The three classes are Claude, GPT, and Qwen, developed by Anthropic, OpenAI, and Alibaba, respectively. Each class includes multiple model versions. The task is family-level classification and does not assume that the responses contain intentional watermarks.

The experiments address two research questions:

- **RQ1:** Can the originating LLM family be identified from its generated response?
- **RQ2:** Does including the user's prompt improve identification?

A convolutional neural network (CNN) and a bidirectional LSTM (BiLSTM) were each trained using prompts only, responses only, and prompts plus responses. All six configurations used the same examples and split assignments.

1.1 Source, Extraction, and Filtering

The data came from the publicly released `lmarena-ai/arena-expert-5k` dataset on Hugging Face [1]. Existing conversations were used; no new responses were generated through model APIs. The loaded dataset contained 5,128 source rows and 105 distinct model identifiers. The loading code did not specify a fixed dataset version.

Conversation fields were stored as string representations of NumPy arrays. A parser reconstructed their structure, and checks confirmed that the recovered text matched the source strings. Each eligible side of a row contributed its final user prompt and assistant response. Responses from `conversation_a` and `conversation_b` were attributed to `model_a` and `model_b`, respectively. Earlier turns were excluded from classifier inputs because their responses could not safely be assigned the final model's label.

An explicit mapping included 13 Claude, 12 GPT, and 13 Qwen identifiers. Other identifiers were excluded, including OpenAI o-series, `gpt-oss`, QwQ, Qwen vision-language variants, and identifiers with an undocumented `-invalid` suffix. Only rows labeled `en` by the publisher were retained, and each selected prompt and response had to contain a single non-empty text item. One otherwise eligible Qwen response had missing content and was excluded.

Filtering produced 2,567 eligible pairs. Removing one exact duplicate, identified by model identifier, prompt, and response, left 2,566 examples: 1,138 Claude, 654 GPT, and 774 Qwen.

1.2 Grouping, Balancing, and Splitting

Before balancing, source rows were grouped whenever they shared an exact non-empty user prompt, including prompts in conversation history. Connections were applied transitively across all source rows, including rows later excluded from classification. Entire groups were kept within a single split. Checks confirmed that neither groups nor exact prompts crossed split boundaries.

Each family was balanced to 654 examples by retaining all GPT examples and sampling Claude and Qwen without replacement using seed 42. This produced 1,962 examples and discarded 604 otherwise usable examples. A deterministic grouped assignment targeted an 80/10/10 distribution within each family, assigning larger groups first and using seeded tie-breaking.

Table 1: Final dataset counts by family and split.

Family	Training	Validation	Test	Total
Claude	522	66	66	654
GPT	522	66	66	654
Qwen	522	66	66	654
Total	**1,566**	**198**	**198**	**1,962**

The selected examples, identifiers, text fingerprints, and split assignments were recorded in `split_reference.csv`. The Colab run reproduced all 1,962 entries exactly.

1.3 Data Limitations and Use

The publisher's English label applies to whole rows, so individual examples may contain other languages. Exact-prompt grouping does not detect paraphrases, and excluding history can remove context needed to interpret a final response. Model versions are also unequally represented within each family.

The project used an existing public dataset without adding private conversations or personally collected interactions. This report presents aggregate results rather than individual conversation text; use and redistribution of source data remain subject to the dataset's applicable terms.

2 Model Implementation and Training

2.1 Tokenization and Input Conditions

The tokenizer preserves capitalization, separates punctuation, groups consecutive word characters, and represents line breaks with `<nl>`. Other whitespace, including indentation, is discarded. Because word matching is Unicode-aware, a continuous run of Chinese, Japanese, or Korean characters may form one token.

A shared vocabulary was built from training prompts and responses only, with no size cap or minimum-frequency threshold. It contains 90,131 entries, including `<pad>`, `<unk>`, `<nl>`, and `<sep>`. Unseen validation and test tokens map to `<unk>`. Both architectures learn 128-dimensional embeddings from scratch.

The three input conditions are:

- **Input only:** the final user prompt.
- **Output only:** the final assistant response.

- **Input + output:** prompt tokens, followed by `<sep>`, followed by response tokens.

Full sequences were retained without truncation. The longest combined training example contained 41,447 tokens.

2.2 CNN and BiLSTM Architectures

CNN. The embedding layer feeds four parallel one-dimensional convolutions with window sizes 2, 3, 4, and 5. Each convolution has 128 filters, stride 1, and no convolutional padding. ReLU activation and global max pooling produce one 128-dimensional vector per window size. The four vectors are concatenated into 512 features, followed by dropout with probability 0.5 and a linear layer producing three class logits. This architecture learns local token patterns and retains their strongest activations across the sequence.

BiLSTM. The embeddings feed a single bidirectional LSTM layer with 128 hidden units per direction. The final forward and backward hidden states are concatenated into 256 features, followed by dropout with probability 0.5 and a linear layer producing three class logits. There is no recurrent dropout. Bidirectionality allows the model to summarize the text in both directions, which is appropriate because the full sequence is available during classification.

2.3 Padding and Sequence Handling

Each batch is right-padded to its longest sequence, with a minimum length of five tokens for the largest CNN window. Padding embeddings remain zero.

The CNN masks convolution positions that extend beyond an example's valid tokens before max pooling. For sequences shorter than a convolution window, one zero-padded window is retained. The BiLSTM uses packed sequences with the original lengths, preventing added padding from entering the recurrent computation. Synthetic checks verified padding invariance for the tested inputs.

2.4 Training and Evaluation

Table 2: Settings shared across all six configurations.

Setting	Value
Batch size	16
Epochs	15
Optimizer	Adam
Learning rate	0.001
Loss	Multiclass cross-entropy
Random seed	42
Embedding dimension	128
Classifier dropout	0.5
Sequence truncation	None

Each configuration began with reset random seeds and a fresh model and optimizer. Training examples were shuffled each epoch. Validation followed every epoch without shuffling, gradient updates, or dropout. Average loss was weighted by batch size.

Accuracy measures the fraction of correct predictions. Macro F1 gives equal weight to the three families:

$$\text{Accuracy} = \frac{\text{correct predictions}}{N}, \qquad \text{Macro F1} = \frac{1}{3}\sum_{c=1}^{3}\frac{2TP_c}{2TP_c + FP_c + FN_c},$$

where TP_c, FP_c, and FN_c are the true positives, false positives, and false negatives for family c. An F1 term with a zero denominator is set to zero.

All configurations completed 15 epochs. The selected checkpoint had the highest validation macro F1, with lower validation loss breaking exact ties. Test evaluation used those checkpoints only after all six runs passed completion and configuration checks. Test results did not select checkpoints or change training settings.

2.5 Execution and Checkpoints

Training ran on a Google Colab Tesla T4 GPU, with logs, metric histories, and checkpoints stored on mounted Google Drive. Checkpoints saved model and optimizer states, progress, configuration and data identities, and random-number states. The latest and best checkpoints were saved separately, with additional checkpoints at epochs 5, 10, and 15.

A GPU save-and-resume self-check passed before real training. The combined-input BiLSTM was then continued from epoch 1 through epoch 15. Its training time was approximately 51–57 seconds per epoch, with 2–3 seconds for validation. The highest logged peak allocated GPU memory was 1.29 GiB. Timings used CUDA synchronization and excluded checkpoint writing.

3 Results

3.1 Performance Across Configurations

Table 3 reports performance on the balanced 198-example test set. Uniform random guessing has an expected accuracy of 33.33%. The output-only CNN achieved the highest accuracy and macro F1.

Table 3: Test performance using validation-selected checkpoints. Bold values identify the highest test scores.

Model	Input condition	Selected epoch	Accuracy	Macro F1
CNN	Input only	13	36.36%	0.3579
CNN	Output only	14	**78.79%**	**0.7856**
CNN	Input + output	15	74.75%	0.7447
BiLSTM	Input only	1	38.89%	0.3888
BiLSTM	Output only	2	65.66%	0.6486
BiLSTM	Input + output	9	55.56%	0.5542

3.2 Training and Validation Curves

Figures 1–6 show loss, accuracy, and macro F1 across training. Dashed lines mark the selected epochs. Training metrics were collected while parameters changed during each epoch; validation metrics used the fixed model afterward.

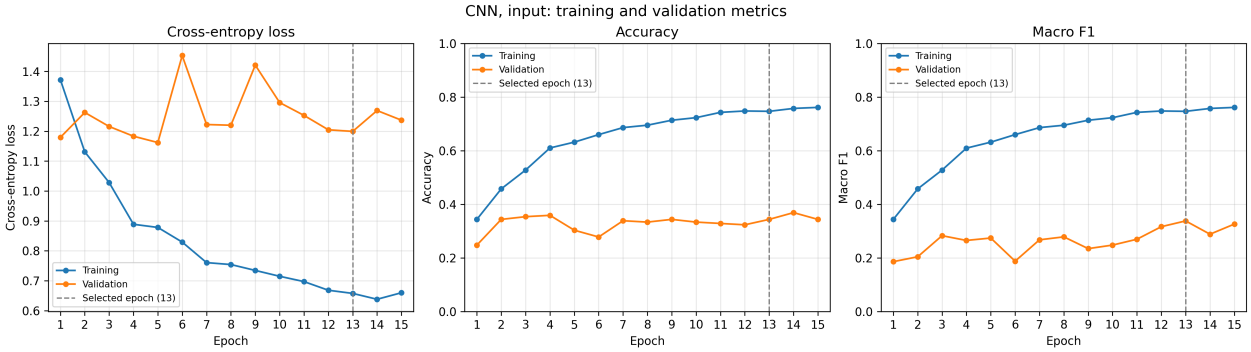


Figure 1: CNN learning curves: input only.

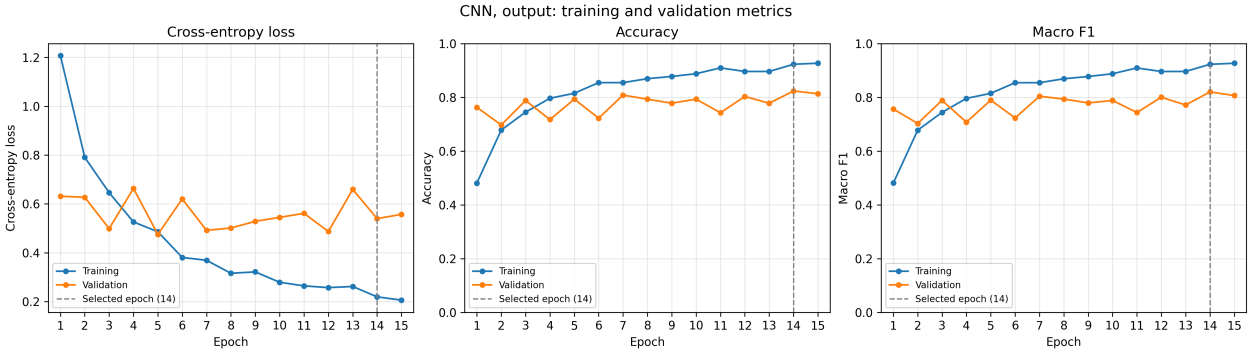


Figure 2: CNN learning curves: output only.

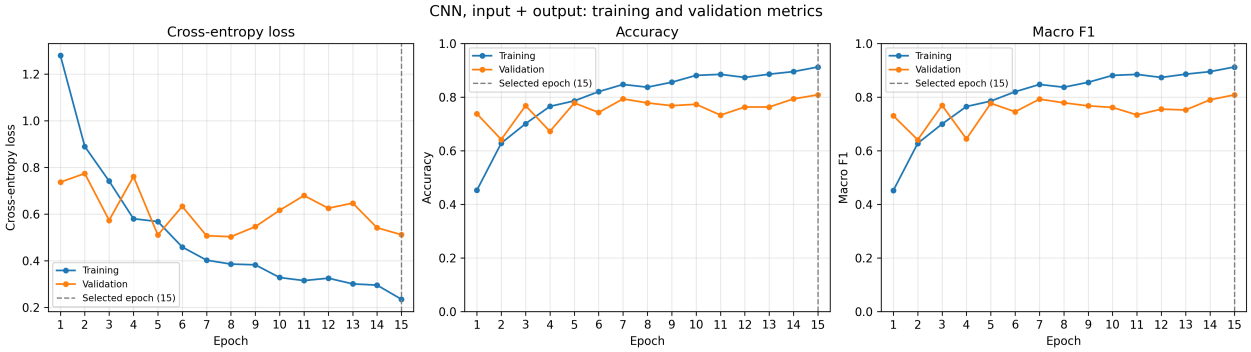


Figure 3: CNN learning curves: input + output.

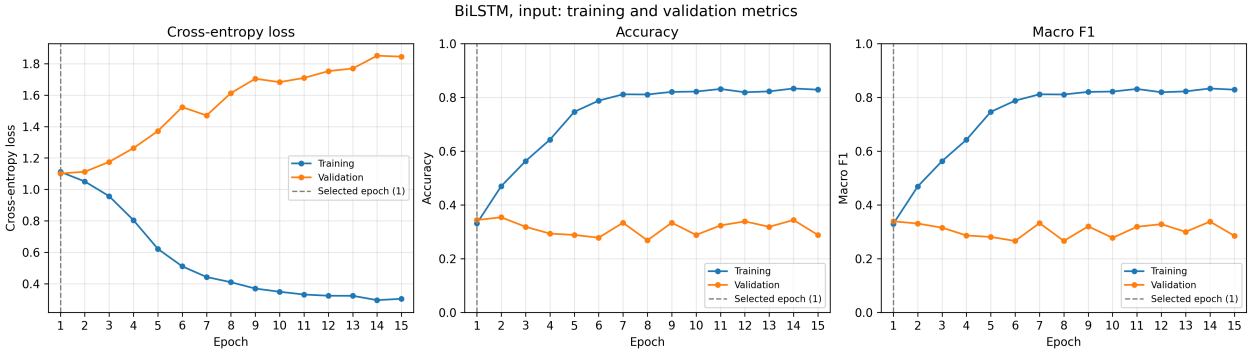


Figure 4: BiLSTM learning curves: input only.

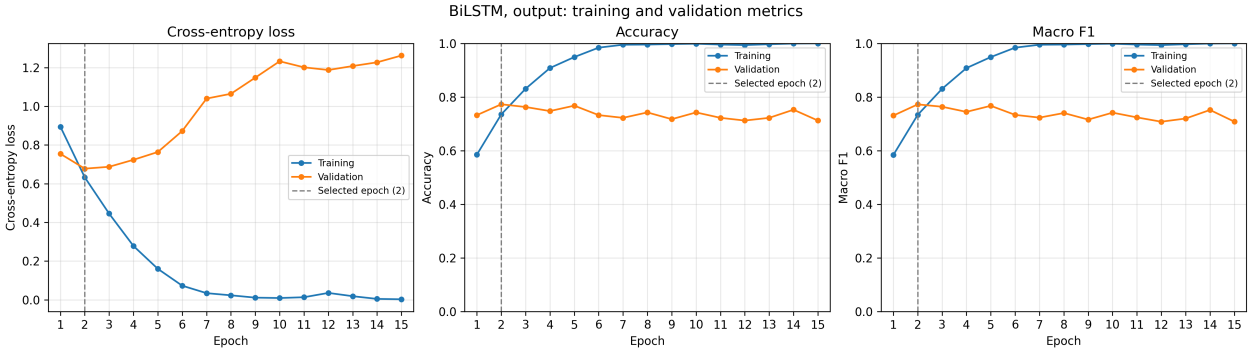


Figure 5: BiLSTM learning curves: output only.

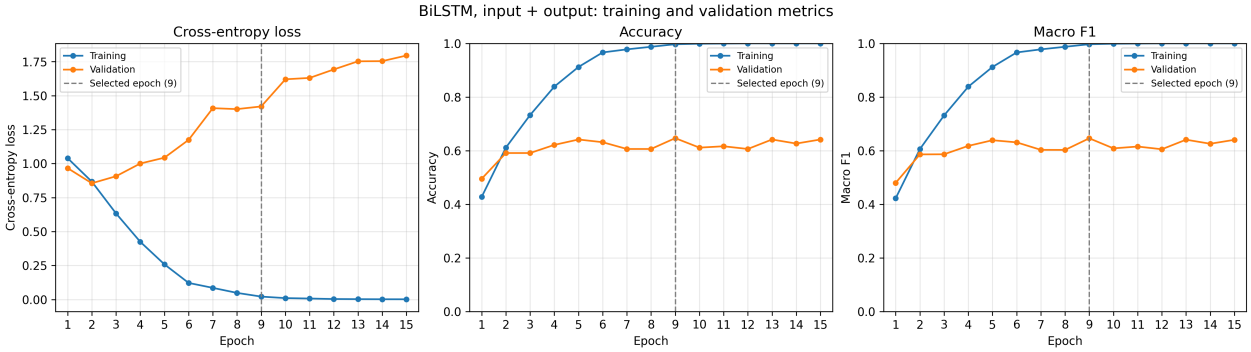


Figure 6: BiLSTM learning curves: input + output.

3.3 Confusion Matrices

Figures 7–12 show test predictions as counts and row-normalized percentages. Rows are true families and columns are predictions, with 66 examples per true family. Diagonal percentages represent recall.

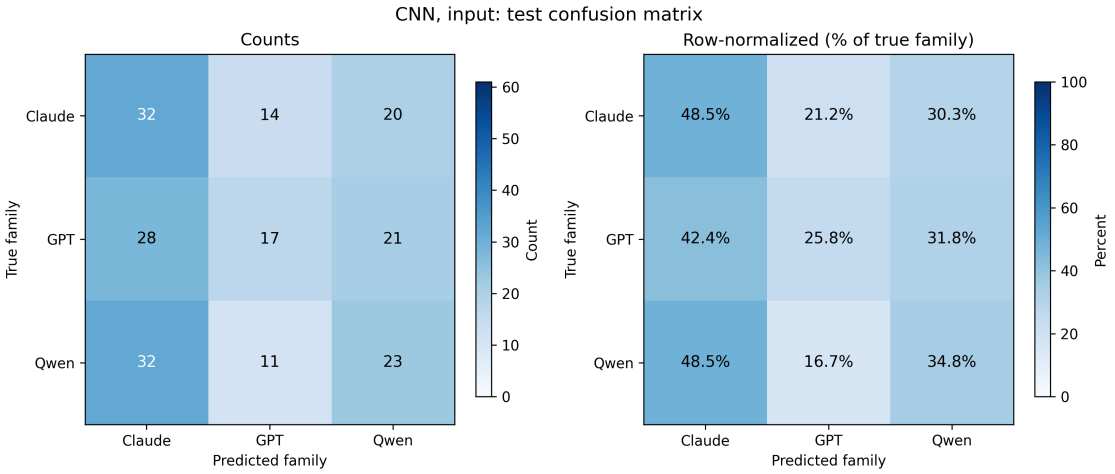


Figure 7: CNN test confusion matrices: input only.

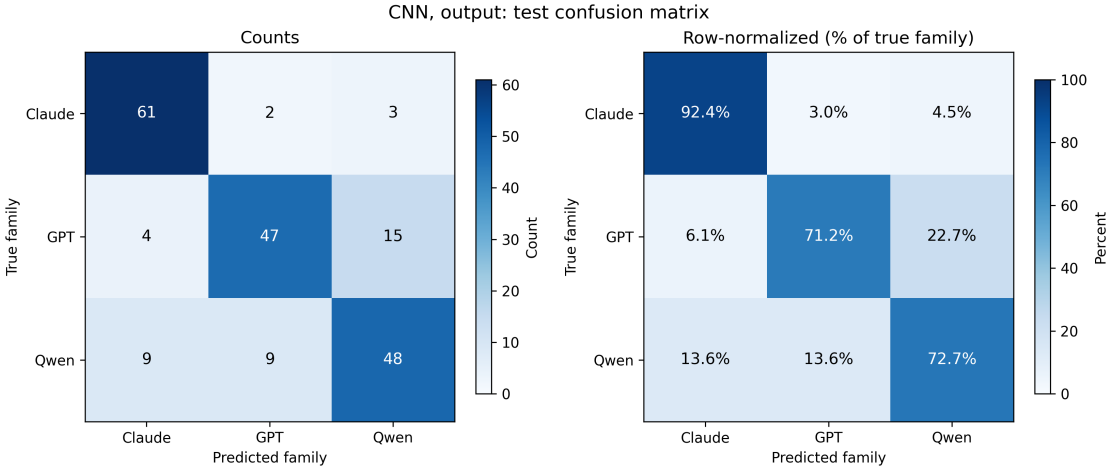


Figure 8: CNN test confusion matrices: output only.

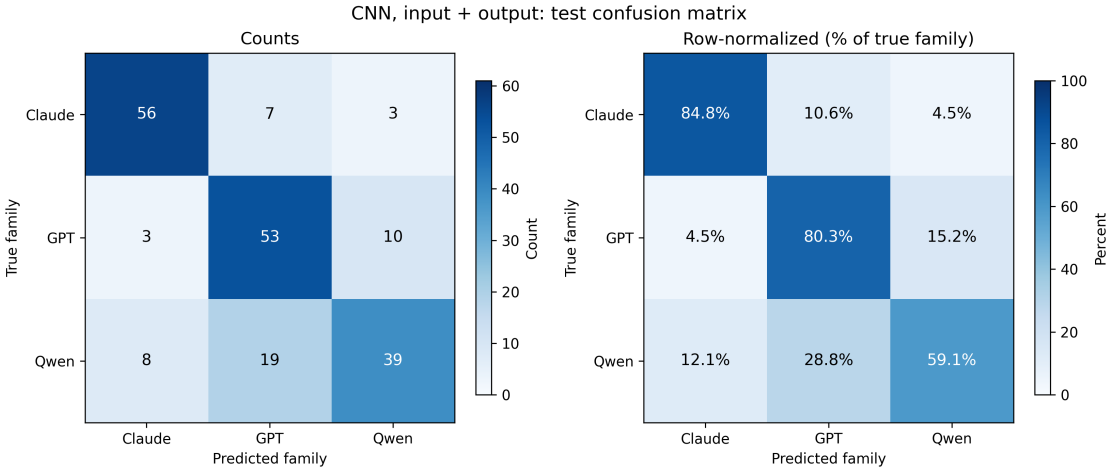


Figure 9: CNN test confusion matrices: input + output.

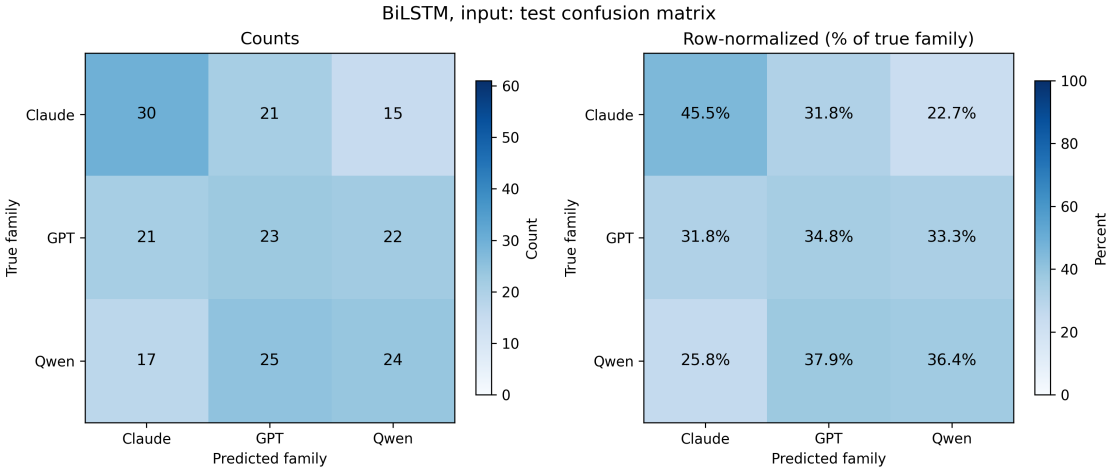


Figure 10: BiLSTM test confusion matrices: input only.

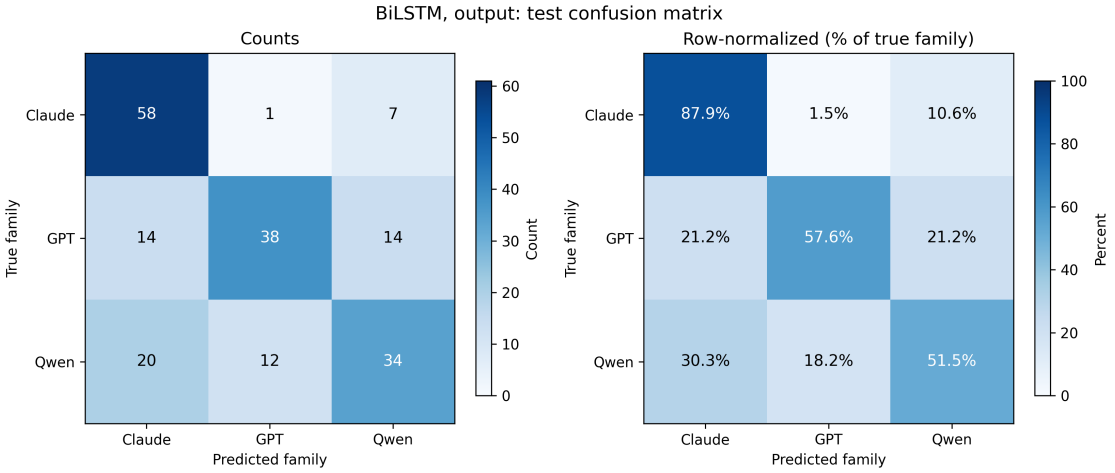


Figure 11: BiLSTM test confusion matrices: output only.

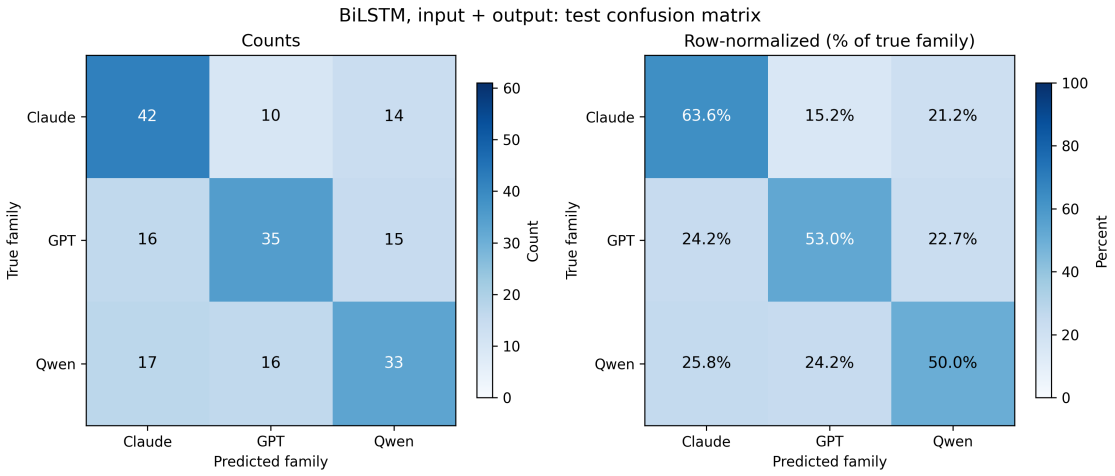


Figure 12: BiLSTM test confusion matrices: input + output.

4 Analysis and Discussion

4.1 RQ1: Can Responses Identify Their Originating Family?

Both output-only models exceeded the 33.33% chance baseline. The CNN correctly classified 156 of 198 responses, compared with 130 for the BiLSTM. This supports family identification within the curated dataset.

The CNN exceeded the BiLSTM by 13.13 percentage points in output-only accuracy and 19.19 points with combined inputs. The BiLSTM held a smaller 2.53-point advantage on prompts alone. Thus, the CNN was stronger when generated responses were available. Local patterns may have helped its convolutions, but no feature-removal experiment established which text characteristics caused this advantage.

4.2 RQ2: Does Including the Prompt Help?

Adding prompts reduced CNN accuracy by 4.04 percentage points and macro F1 by 0.0409. For the BiLSTM, accuracy fell by 10.10 points and macro F1 by 0.0944. Responses alone therefore worked best for both architectures under the tested settings.

Input-only accuracy exceeded chance by only 3.03 points for the CNN and 5.56 points for the BiLSTM. These modest gains provide limited evidence of useful prompt information. Any association could reflect topic or request patterns in the source data rather than properties of generated text.

Prompts may introduce less relevant information or make the sequences harder to learn from by increasing their length. These remain possible explanations, not established causes. The results do not show that prompts always harm classification.

4.3 Family-Level Errors

The output-only CNN had its highest recall for Claude: 61 of 66 responses were correct, compared with 47 GPT and 48 Qwen responses. Confusion between GPT and Qwen accounted for 24 of its 42 errors, including 15 GPT responses classified as Qwen.

The output-only BiLSTM also recognized Claude most often, with 58 correct predictions, compared with 38 for GPT and 34 for Qwen. Its largest error category was Qwen classified as Claude, occurring 20 times. It predicted Claude for 92 examples despite the balanced labels, so strong Claude recall coincided with overprediction of that class.

Adding prompts affected families differently. For the CNN, correct GPT predictions rose from 47 to 53, while Claude fell from 61 to 56 and Qwen from 48 to 39. The overall decline therefore concealed an improvement for GPT. For the BiLSTM, correct predictions decreased for every family; Claude accounted for 16 of the 20 fewer correct classifications.

4.4 Overfitting and Validation-to-Test Performance

The combined-input BiLSTM provides a clear example of overfitting. Training accuracy reached 100% by epoch 12, while later validation accuracy remained around 60–65%. Validation loss increased from 0.8554 at epoch 2 to 1.7950 at epoch 15, as training loss fell to 0.0016.

Epoch 9 was selected because it had the highest validation macro F1, approximately 0.646. Its test macro F1 was 0.5542, and accuracy fell from approximately 64.6% on validation to 55.56% on test. This gap shows that validation-based selection does not guarantee the same performance on unseen examples.

The first two BiLSTM configurations selected epochs 1 and 2, while the CNN configurations selected epochs 13–15. Training longer therefore did not consistently produce the best validation checkpoint. Test results were reported for the selected checkpoints without choosing replacement epochs.

4.5 Scope of the Findings

The experiment used one split and one training seed. With 198 test examples, one prediction changes accuracy by approximately 0.51 percentage points. The results do not measure variability across repeated runs, and version or topic differences may affect performance.

These findings support RQ1 and RQ2 within this dataset. They do not establish generalization across domains or identify the linguistic features responsible for predictions. The optional cross-domain and feature-analysis research questions were not evaluated.

5 Lessons and Experience

5.1 Data Preparation and Reproducibility

The most important data lesson was that valid text and valid labels require separate checks. Parsing had to preserve the original strings, while attribution required distinguishing final responses from earlier conversation history. Explicit family mappings made ambiguous identifiers easier to handle consistently.

Deduplication alone was insufficient for leakage prevention: different responses could share a prompt. Grouping before splitting addressed this issue, and the split-reference file provided a direct reproducibility check across environments. Balancing simplified comparisons but also demonstrated the cost of discarding usable examples.

5.2 Sequence Handling and GPU Execution

Padding is part of model correctness, not just batch formatting. The CNN required masking and the BiLSTM required packing so that batch composition would not introduce artificial text features.

GPU benchmarks helped assess whether full sequences were practical before changing the input representation. The first complete epoch then measured realistic shuffled batches, whose costs differed from fixed benchmark batches. This experience showed the value of measuring resource use before imposing a truncation limit.

5.3 Persistence and Continuation

Colab required explicit checks that Drive was mounted before saving outputs. An existing directory alone did not establish that files would persist after the runtime ended.

Reliable continuation required optimizer and random-number states as well as model weights. GPU verification also caught a device-specific issue that CPU testing could not establish: checkpoint loading had to preserve random-number states as CPU tensors. Loading checkpoints on the CPU first, followed by the GPU resume self-check, verified the continuation path before the full experiment.

5.4 Interpreting Performance

The experiments reinforced the distinction between fitting training data and generalizing to new examples. Validation-based selection prevented automatic use of the final epoch, while the separate test set exposed performance differences that training metrics could not show.

More input text did not necessarily help: adding prompts reduced overall performance, but the family-level errors revealed a more nuanced effect for the CNN. Learning curves, aggregate scores, and confusion matrices each contributed information needed to interpret the models fairly.

References

[1] LMArena. *Arena Expert 5K*. Hugging Face dataset repository. `https://huggingface.co/datasets/lmarena-ai/arena-expert-5k`. Accessed October 11, 2026.